

Polytechnique Montréal
Département de génie informatique et génie logiciel

Cours INF1900:
Projet initial de système embarqué

Travail pratique 7
Makefile et production de librairie statique

Par l'équipe
No 1621

Noms:
Anis Feteḥ Benabderrahmane
Giovanni-Marven Alcena
Elliott Gareau
Adam Azrou

Date:
17 mars 2025

Partie 1 : Description de la librairie

Notre librairie est constituée de plusieurs classes. Nous décrirons ces classes en nous appuyant sur leurs objectifs, leurs constructeurs et les méthodes implémentées dans les lignes qui suivent :

Classes Timer 0, Timer 1 et Timer 2 des fichiers « timer0.h », « timer1.h » et « timer2.h »

Objectifs :

Les classes Timer0 et Timer2 permettent de configurer et de contrôler les timers 8 bits du microcontrôleur ATmega324PA. La classe Timer1 permet de configurer et de contrôler le timer 16 bits du microcontrôleur. Les Timer0 et Timer2 sont pour le PWM et le Timer1 pour le délai.

Constructeurs :

Timer0(TimerMode mode) : Initialise le Timer0 avec le mode spécifié, un prescaler par défaut, et désactive les interruptions.

Timer2(TimerMode mode) : Initialise le Timer2 avec les mêmes paramètres.

Timer1(TimerMode mode) : Initialise le Timer1 avec le mode spécifié, un prescaler par défaut, et désactive les interruptions. Il inclut également la gestion d'une variable d'expiration.

Méthodes implémentées communes aux trois classes :

- void setPrescaler(Prescaler value): Configure le prescaler du timer.
- void setTimerMode(TimerMode mode): Définit le mode de fonctionnement (Normal, CTC, PWM).
- void setCompareOutputModeA(CompareOutputMode mode): Configure le mode de sortie sur la broche de comparaison A.
- void setCompareOutputModeB(CompareOutputMode mode): Configure le mode de sortie sur la broche de comparaison B.
- void setCompareValue(OutputComparePin pin, uint8_t ou uint16_t value): Définit la valeur de comparaison OCRxA et OCRxB.
- void setTimerValue(uint8_t ou uint16_t value): Initialise la valeur du compteur TCNTx.
- void allowInterrupts(OutputComparePin pin): Active ou désactive les interruptions pour les broches de comparaison.

Méthodes implémentées uniquement pour Timer1 :

- void initializeTimerForDelays(volatile bool& glsExpired): Initialise un timer pour la gestion des délais.
- void startTimer(uint16_t calculatedDelay): Démarre le timer avec une valeur de délai.
- bool isExpired(): Retourne l'état du timer pour indiquer si le délai est atteint.

Class Memoire24 du fichier memoire_24.h

Objectifs :

Il s'agit d'une classe qui a pour rôle de permettre à l'utilisateur de manipuler la mémoire externe de notre robot via l'interface I2C grâce aux ports C0 et C1.

Constructeurs :

Memoire24CXXX(): Initialise la mémoire avec la taille de page et configure l'interface I2C.

Méthodes implémentées (communes aux deux classes) :

- void init(): Initialise l'horloge de l'interface I2C et configure le banc de mémoire.
- uint8_t choisir_banc(const uint8_t banc): Permet de sélectionner un banc de mémoire.
- uint8_t lecture(const uint16_t adresse, uint8_t *donnee): Lit une donnée unique à une adresse mémoire.
- uint8_t lecture(const uint16_t adresse, uint8_t *donnee, uint8_t longueur): Lit un bloc de données à partir d'une adresse mémoire.
- uint8_t ecriture(const uint16_t adresse, const uint8_t donnee): Écrit une donnée unique à une adresse mémoire.
- uint8_t ecriture(const uint16_t adresse, uint8_t *donnee, const uint8_t longueur): Écrit un bloc de données en mode page.

Classe Led du fichier « led.h »

Objectif : La classe Led permet de contrôler l'état d'une LED bicolore en utilisant deux broches du microcontrôleur ATmega324PA.

Constructeur :

Led(Ports *portsUtility, Port port, Pin pinA, Pin pinB): Initialise la LED en configurant les broches associées en mode sortie.

Méthodes implémentées :

- void lightUp(Color color): Change l'état de la LED selon la couleur spécifiée (VERT, ROUGE ou ÉTEINT).
- ~Led(): Désactive la LED en reconfigurant les broches en mode lecture.

Classe Can du fichier « can.h »

Objectif : La classe Can permet d'accéder au convertisseur analogique-numérique (CAN) du microcontrôleur ATmega324PA pour lire des valeurs analogiques depuis un capteur.

Constructeur :

- `can()`: Initialise le convertisseur analogique-numérique (ADC) en configurant ses registres pour une conversion standard avec une référence analogique externe.

Méthodes implémentées :

- `uint16_t lecture(uint8_t pos)`: Effectue une conversion analogique-numérique sur la broche spécifiée et retourne une valeur 16 bits, dont seuls les 10 bits de poids faibles sont significatifs.
- `~can()`: Désactive le convertisseur ADC pour réduire la consommation d'énergie.

Classe Button du fichier « button.h »

Objectif :

La classe Button permet de gérer l'état d'un bouton poussoir, en prenant en charge les interruptions externes si nécessaire. Elle optimise les performances puisqu'elle utilise les interruptions au lieu de la scrutation.

Constructeur :

`Button(ButtonType type, volatile bool *gButton)`: Initialise le bouton en définissant son type (externe ou interne) et configure l'interruption externe si nécessaire.

Méthodes implémentées :

- `void activateExternalInterrupt()`: Active l'interruption externe INT0 pour détecter les pressions sur un bouton externe.
- `bool isPressed()`: Retourne l'état du bouton (pressé ou non), en fonction de son type.

Classe Ports du fichier « ports.h »

Objectif :

La classe Ports permet de gérer les entrées et sorties numériques des ports du microcontrôleur ATmega324PA.

Méthodes implémentées :

- `void setPinMode(PortMode mode, Port port, Pin pin)`: Configure le mode d'un pin (lecture ou écriture) pour un port donné.
- `uint8_t readPort(Port port)`: Lit l'état de l'ensemble des pins d'un port spécifique.
- `void writePort(Port port, uint8_t value)`: Écrit une valeur sur l'ensemble des pins d'un port spécifique.

- bool readPin(Port port, Pin pin): Lit l'état d'un pin spécifique d'un port donné.
- void writePin(Port port, Pin pin, bool value): Écrit un état logique (haut ou bas) sur un pin spécifique d'un port donné.

Classe DEBUG du fichier « debug.h »

Objectif : La classe DEBUG permet d'envoyer des messages de débogage via l'interface RS232.

Méthodes implémentées :

- void display(const char* message): Envoie un message texte via l'interface RS232.
- void display(uint16_t x): Convertit un entier en chaîne de caractères et l'envoie via l'interface RS232.

Classe RS232 du fichier « rs232.h »

Objectif : La classe RS232 permet d'envoyer des données via l'interface série USART du microcontrôleur ATmega324PA.

Constructeur :

- RS232(): Initialise la communication série en configurant les registres USART pour un format de trame de 8 bits, 1 bit de stop et sans parité.

Méthodes implémentées :

- void write(const char* message): Envoie une chaîne de caractères via l'interface RS232 en s'assurant que le registre de transmission est prêt à recevoir de nouvelles données.

Classe Wheels du fichier « wheels.h »

Objectif : La classe Wheels permet de contrôler les roues d'un robot en utilisant le Timer2 pour la gestion du PWM et le Timer1 pour les délais.

Constructeur :

Wheels(Timer1 *delayTimer, Timer2 *pwmTimer, Ports *portsUtility): Initialise les roues en configurant les broches nécessaires en sortie et en paramétrant le Timer2 en mode PWM.

Méthodes implémentées :

- void rotateLeftWheel(uint8_t pourcentage, Direction direction): Active la roue gauche avec une puissance spécifiée et une direction (avant ou arrière).
- void rotateRightWheel(uint8_t pourcentage, Direction direction): Active la roue droite avec une puissance spécifiée et une direction (avant ou arrière).

Partie 2 : Description des Makefiles

Makefile de la librairie :

Objectif : Ce Makefile permet de compiler et d'assembler une bibliothèque statique. Il génère la bibliothèque **libstatic.a** et gère la compilation des fichiers sources en respectant les dépendances et les optimisations nécessaires.

- PROJECTNAME=libstatic : Nom du projet.
- PRJSRC=\$(wildcard *.cpp) : Récupère tous les fichiers sources C++ du répertoire.
- LIBTRG=\$(PROJECTNAME).a : Définit le nom de la bibliothèque générée.
- AR=ar : Utilisé pour assembler les fichiers objets en une bibliothèque statique.
- \$(LIBTRG): \$(OBJDEPS) : Regroupe les fichiers objets (.o) en une bibliothèque statique avec ar -crs.
- all: \$(LIBTRG) \$(TRG) \$(HEXROMTRG) : Ajout de libtrg compile la bibliothèque statique
- clean: \$(REMOVE) \$(LIBTRG) \$(TRG) \$(TRG).map \$(OBJDEPS) \$(HEXTRG) *.d : Enlève la bibliothèque statique.

Makefile de l'exécutable :

- PROJECTNAME=makefileExec : Nom du projet.
- PRJSRC=\$(wildcard *.cpp) : Récupère tous les fichiers sources C++ du répertoire.
- LIBS=-L../-llibstatic : Lien avec la bibliothèque statique **libstatic.a**.
- INC= -I../lib : Commande pour spécifier le chemin des inclusions additionnelles.